

LAB 6 PROBLEM SOLVING IMPLEMENTING CRUD OPERATIONS OF A SYSTEM

Course: System Development Course Code: 420-940-VA Week: 6 Lab Duration: 5 hours

INTRODUCTION

By Week 6 the project must move beyond screens and buttons. Week 5 establishes the first user interface prototype. Week 6 requires the prototype to manage data in a structured way. CRUD means Create, Read, Update, and Delete. These are the four most common operations performed on records in an information system. For an advising system, CRUD operations create appointment records, read them back, update them, and delete or cancel them. For an inventory system, CRUD operations create product records, read product information, update stock or details, and delete records when needed.

This lab implements CRUD in a realistic first version of the project using C# and the existing interface work. Because Week 8 is the part of the course where the outline explicitly mentions implementing the system database using SQL Server, this Week 6 lab focuses on CRUD logic and form behavior first. In other words, this lab practises the working logic of CRUD in the system now while still preparing for stronger database integration later.

Main Focus of This Lab

-  Selection of one main entity from the project
-  Implementation of Create, Read, Update, and Delete behavior
-  Connection of Windows Forms controls to project data
-  Validation of input
-  Display of records clearly in the interface
-  Preparation of a strong technical foundation for Assignment 4

Lab Objectives Participants achieve the following:

- Identification of one main project entity that can support CRUD operations
- Creation of a simple data model for that entity
- Construction of a Windows Forms screen that supports Create, Read, Update, and Delete
- Validation of user input before saving changes
- Updating and removal of records from the system interface
- Explanation of how the CRUD work connects to the database design from Chapter 6
- Preparation of the implemented CRUD component required for Week 6 lab work and Assignment 4

Required Materials

-  Access to Omnivox
-  Course work from Weeks 1 to 5
-  Database design notes from Chapter 6
-  Interface prototype from Week 5
-  Visual Studio with C# Windows Forms support
-  Word processor (Microsoft Word or Google Docs)
-  Screenshot tool

Important Week 6 Note

This lab addresses the exact Week 6 lab direction in the outline: problem solving implementing CRUD Operations of a System. Since the outline names Implementing Database of System using SQL Server in Week 8, this lab does not require completion of the full SQL Server stage yet unless the instructor specifically asks for it. The lab builds the CRUD logic and form behavior in a clean, structured way so that the project is ready for stronger database integration later.

Suggested Submission Package One compressed file is uploaded to Omnivox: **Lab6_CRUD_LastName.zip**

The compressed file contains:

1. The complete Visual Studio project folder
2. The lab document named **Lab6_CRUD_LastName.docx**

The lab document contains:

- ✚ Project and entity summary
- ✚ Screenshots of the working CRUD interface
- ✚ Key code sections
- ✚ Explanation of Create, Read, Update, and Delete
- ✚ Testing notes
- ✚ Reflection answers

Lab Structure and Timing The lab is structured as a full 5-hour session.

PHASE 1: REVIEW OF PROJECT AND SELECTION OF ONE MAIN ENTITY TIME: 25 MINUTES

Previous project work is opened:

- ✚ Week 2 requirement document
- ✚ Week 3 user stories
- ✚ Week 4 UML diagrams
- ✚ Week 5 interface prototype

One main entity from the project is chosen for CRUD implementation. Examples: Appointment, Ticket, Product, EventRegistration, StudentRequest, InventoryItem.

A section titled **1. Project and Entity Summary** is created in the lab document. One paragraph explains:

- ✚ The project name
- ✚ The selected entity
- ✚ Why this entity is a good choice for CRUD in Week 6
- ✚ The fields needed for that entity

For example, if the entity is Appointment, the fields might include: AppointmentID, StudentNumber, AppointmentDate, Reason, Status.

Purpose of this phase CRUD implementation remains based on a real project object.

PHASE 2: PLANNING OF THE CRUD FORM AND DATA STRUCTURE TIME: 35 MINUTES

A section titled **2. CRUD Form Plan** is created in the lab document. The plan describes:

- ✚ Controls the form will contain
- ✚ Required fields
- ✚ How records will be displayed
- ✚ How the user will select a record for updating or deleting

A strong beginner layout usually includes: title label, text boxes for key fields, a DataGridView, Create button, Update button, Delete button, and Clear button.

A short note explains how the form supports Create, Read, Update, and Delete.

Purpose of this phase Interface design is connected to data behavior before implementation starts.

PHASE 3: CREATION OF THE MODEL CLASS AND TEMPORARY DATA SOURCE TIME: 50 MINUTES

Visual Studio a simple C# class is created for the entity. For example, if the entity is Appointment, a file called Appointment.cs is created.

The following sample model is used:

C#

```
public class Appointment // This line creates a class called Appointment. A class is a blueprint for objects of the same type.
{
    public int AppointmentID { get; set; } // This property stores the unique identifier for one appointment.

    public string StudentNumber { get; set; } // This property stores the student's number as text.

    public string AppointmentDate { get; set; } // This property stores the appointment date. In a more advanced version, this
    could later become a true DateTime type.

    public string Reason { get; set; } // This property stores the reason for the appointment.

    public string Status { get; set; } // This property stores the current status, such as Requested or Approved.
}
```

In the form code a temporary collection is created to hold records while the program runs:

C#

```
private List<Appointment> appointments = new List<Appointment>(); // This line creates an empty list that will temporarily
store Appointment objects in memory while the program is running.
```

A section titled **3. Data Model and Temporary Storage** is created in the lab document. The section explains:

- ❖ The class created
- ❖ What the properties mean
- ❖ Why a temporary in-memory list is used in this week's lab
- ❖ How this prepares the project for stronger database work later

Purpose of this phase The system receives a structured data model before CRUD behavior is implemented.

PHASE 4: IMPLEMENTATION OF CREATE AND READ TIME: 70 MINUTES

The form is made to support the first two CRUD actions: Create and Read.

Part A: Create The following sample Create button method is adapted:

C#

```
private void btnCreate_Click(object sender, EventArgs e) // This method runs when the user clicks the Create button.
{
    if (txtStudentNumber.Text == "" || txtDate.Text == "" || txtReason.Text == "") // This line checks whether any required field is
        empty.
    {
        MessageBox.Show("Please complete all required fields before creating the record.", "Missing Information"); // This line
        shows a warning message to the user.

        return; // This line stops the method so no record is created when required fields are missing.
    }

    Appointment newAppointment = new Appointment(); // This line creates a new Appointment object based on the class
    blueprint.

    newAppointment.AppointmentID = appointments.Count + 1; // This line assigns a simple ID by counting how many records
    are currently in the list and adding one.

    newAppointment.StudentNumber = txtStudentNumber.Text; // This line copies the student number from the text box into the
    new object.

    newAppointment.AppointmentDate = txtDate.Text; // This line copies the appointment date into the new object.

    newAppointment.Reason = txtReason.Text; // This line copies the reason into the new object.

    newAppointment.Status = txtStatus.Text; // This line copies the status into the new object.

    appointments.Add(newAppointment); // This line adds the new object to the appointments list.

    RefreshGrid(); // This line updates the DataGridView so the new record becomes visible.

    ClearFields(); // This line clears the input controls after the record is created.
```

```

        MessageBox.Show("Record created successfully.", "Success"); // This line shows a success message.
    }

```

Part B: Read A helper method is created to display records:

C#

```

private void RefreshGrid() // This method refreshes the DataGridView so it always shows the current records.
{
    dgvAppointments.DataSource = null; // This line clears the current data binding from the grid so it can be refreshed properly.

    dgvAppointments.DataSource = appointments; // This line binds the grid to the current list of appointment records.
}

```

A helper method to clear the text boxes is also created:

C#

```

private void ClearFields() // This method clears the input controls so the user can enter a new record easily.
{
    txtStudentNumber.Text = ""; // Clears the student number field.

    txtDate.Text = ""; // Clears the date field.

    txtReason.Text = ""; // Clears the reason field.

    txtStatus.Text = ""; // Clears the status field.

    txtStudentNumber.Focus(); // Returns the cursor to the first field.
}

```

A section titled **4. Create and Read Implementation** is created in the lab document. The section explains:

- How the Create action works
- How records are shown in the DataGridView
- What validation was added before creating a record

Purpose of this phase The first working core of CRUD is built.

PHASE 5: IMPLEMENTATION OF UPDATE AND DELETE TIME: 60 MINUTES

The system is made to support Update and Delete.

Part A: Selection of a record from the DataGridView The following method is implemented:

C#

```
private void dgvAppointments_CellClick(object sender, DataGridViewCellEventArgs e) // This method runs when the user clicks
a row in the grid.
```

```
{
    if (e.RowIndex >= 0) // This line checks that the user clicked a real row and not the header row.
    {
        txtStudentNumber.Text = dgvAppointments.Rows[e.RowIndex].Cells["StudentNumber"].Value.ToString(); // Loads the
StudentNumber value into the text box.

        txtDate.Text = dgvAppointments.Rows[e.RowIndex].Cells["AppointmentDate"].Value.ToString(); // Loads the
AppointmentDate value into the text box.

        txtReason.Text = dgvAppointments.Rows[e.RowIndex].Cells["Reason"].Value.ToString(); // Loads the Reason value into the
text box.

        txtStatus.Text = dgvAppointments.Rows[e.RowIndex].Cells["Status"].Value.ToString(); // Loads the Status value into the
text box.
    }
}
```

Part B: Update

C#

```
private void btnUpdate_Click(object sender, EventArgs e) // This method runs when the user clicks the Update button.
{
    if (dgvAppointments.CurrentRow == null) // This line checks whether a row is currently selected.
    {
        MessageBox.Show("Please select a record before updating.", "No Selection"); // Warns the user if nothing is selected.

        return; // Stops the method.
    }
}
```

```
int rowIndex = dgvAppointments.CurrentRow.Index; // This line stores the selected row index.
```

```
appointments[rowIndex].StudentNumber = txtStudentNumber.Text; // Updates the StudentNumber value in the selected object.
```

```
appointments[rowIndex].AppointmentDate = txtDate.Text; // Updates the AppointmentDate value.
```

```
appointments[rowIndex].Reason = txtReason.Text; // Updates the Reason value.
```

```
appointments[rowIndex].Status = txtStatus.Text; // Updates the Status value.
```

```
RefreshGrid(); // Refreshes the grid so the changes appear.
```

```
ClearFields(); // Clears the input controls.
```

```
MessageBox.Show("Record updated successfully.", "Success"); // Shows a success message.
```

```
}
```

Part C: Delete

```
C#
```

```
private void btnDelete_Click(object sender, EventArgs e) // This method runs when the user clicks the Delete button.
```

```
{
```

```
if (dgvAppointments.CurrentRow == null) // This line checks whether a row is selected.
```

```
{
```

```
    MessageBox.Show("Please select a record before deleting.", "No Selection"); // Warns the user if no row is selected.
```

```
    return; // Stops the method.
```

```
}
```

```
int rowIndex = dgvAppointments.CurrentRow.Index; // Stores the selected row index.
```

```
appointments.RemoveAt(rowIndex); // Removes the selected object from the list.
```

```
RefreshGrid(); // Refreshes the grid after deletion.
```

```
ClearFields(); // Clears the form fields.
```

```
MessageBox.Show("Record deleted successfully.", "Success"); // Shows a success message.
```

```
}
```

A section titled **5. Update and Delete Implementation** is created in the lab document. The section explains:

- How the selected record is loaded
- How the Update button changes the record
- How the Delete button removes the record
- What safety checks were used before updating or deleting

Purpose of this phase The four CRUD actions are completed.

PHASE 6: TESTING OF THE FULL CRUD CYCLE AND PREPARATION OF ASSIGNMENT 4
SUPPORT MATERIAL TIME: 60 MINUTES

The program is tested carefully. The full cycle is performed:

1. Creation of at least three records
2. Reading of the records in the DataGridView
3. Updating of at least one record
4. Deletion of at least one record
5. Testing of one invalid case such as empty required fields

Screenshots are taken showing:

-  The CRUD form
-  At least one created record in the grid
-  One validation message
-  One updated record
-  One delete result

A final section titled **6. Testing Results and Assignment 4 Preparation** is created. Short paragraph answers address:

1. The entity implemented and why
2. The part of CRUD that worked best in the project
3. The problem faced while testing
4. What was fixed or improved
5. What would need to change later when the temporary list is replaced with a real database

A short bullet list titled **What can be used in Assignment 4** is added. Items include:

-  Entity class
-  CRUD form
-  Create code
-  Read code
-  Update code
-  Delete code

-  Validation logic
-  Screenshots
-  Testing notes

Purpose of this phase The lab connects directly to Week 6 Assignment 4 and to the later database implementation stage.

What Must Be Submitted One compressed file: **Lab6_CRUD_LastName.zip**

Contents:

-  Complete Visual Studio project folder
-  Lab document **Lab6_CRUD_LastName.docx**

Required Order of Sections in the Lab Document

1. Project and Entity Summary
2. CRUD Form Plan
3. Data Model and Temporary Storage
4. Create and Read Implementation
5. Update and Delete Implementation
6. Testing Results and Assignment 4 Preparation

Sample Mini Model Example entity Appointment

Example fields AppointmentID, StudentNumber, AppointmentDate, Reason, Status

Example Create explanation The Create button reads the user input from the text boxes, checks that the required fields are completed, creates a new Appointment object, adds it to the list, and refreshes the DataGridView.

Example Update explanation The Update button changes the selected record after the user edits the fields. This allows the project to modify existing information instead of creating duplicate records.

Example reflection note While testing the CRUD cycle, it was noticed that updating a record without selecting a row caused confusion. A message box was added to force the user to select a record first. This made the system clearer and more reliable.